

Parallel and Distributed K-Nearest-Neighbor Classification: Comparing POSIX Threads, OpenMP, and MPI

Your Name

Department of Computer Science
Virginia Commonwealth University, VCU
Richmond, USA
yourname@vcu.edu

Abstract—This report presents a serial baseline and three parallel implementations of the k-nearest-neighbor (KNN) classifier: a shared-memory version using POSIX threads, a shared-memory version using OpenMP, and a distributed version using MPI. All three parallel versions partition the classification work along the same axis, the set of test instances, so that every worker reproduces the exact predictions of the serial version regardless of how many workers are used. The implementations were developed and correctness tested on `maple.cs.vcu.edu` and evaluated for runtime and speedup on both `maple.cs.vcu.edu` and the `athena.hprc.vcu.edu` Slurm cluster, using three datasets of increasing size and worker counts of 1, 2, 4, 8, 16, 32, 64, and 128.

Index Terms—HPDS, KNN, POSIX threads, OpenMP, MPI, parallel computing, speedup

I. INTRODUCTION

The k-nearest-neighbor classifier assigns a class to a query instance by finding the k closest instances to it in a labeled training set and taking a majority vote of their labels. Its accuracy is competitive with far more complex classifiers on many problems, but its cost grows as $O(num_test \times num_train \times num_features)$, since every test instance must be compared against every training instance across every feature. On datasets with tens of thousands of instances this cost becomes the practical limit on how quickly the classifier can be run, which makes KNN a useful vehicle for comparing parallelization strategies: the algorithm itself is simple enough that the effect of the parallelization strategy, rather than the complexity of the algorithm, dominates the results.

This report implements and evaluates three parallel strategies for the same classification workload. All three are built around the same partitioning decision, splitting the test instances across workers rather than the training instances, for a reason argued in Section II-B: it is the only one of the two splits that reproduces the serial version’s predictions exactly, including on ties, without any extra work.

The rest of this report is organized as follows. Section II describes the serial algorithm, the shared partitioning strategy, the three parallel implementations, and the experimental setup and metrics used to evaluate them. Section III presents the measured runtimes, speedups, and accuracy. Section IV

discusses the trends in those results and compares the three approaches. Section V concludes.

II. METHODOLOGY

A. Serial KNN Algorithm

The provided serial implementation classifies each test instance independently. For a single test instance, it computes the Euclidean distance to every training instance, maintains a sorted list of the k closest training instances seen so far, and after scanning the full training set predicts the class that appears most often among those k neighbors. A new training instance is inserted into the candidate list only when its distance is strictly smaller than an existing candidate’s distance, so among instances at exactly equal distance, the one encountered earlier in the scan keeps its place. This tie-breaking rule matters for Section II-B.

B. Work Partitioning Strategy

All three parallel implementations divide the set of test instances into contiguous blocks and assign one block to each worker (thread or MPI process), which then scans the entire training set for each test instance in its block, exactly as the serial version does. Worker t of P workers is given the half-open range $[tn/P, (t+1)n/P)$ over n test instances. This splits the test instances into P blocks whose sizes differ by at most one, and it degrades gracefully when P exceeds n : a worker with $t \geq n$ receives an empty range and simply does no work, which occurs in this evaluation whenever the small dataset (160 test instances) is run with more than 160 workers.

An alternative partitioning splits the training set instead, and has every worker scan a shard of the training instances for the same query, building partial candidate lists that are then merged. This exposes more parallel work, since the training sets used here are an order of magnitude larger than the test sets, but it changes the order in which training instances are compared for a given query. Because the serial algorithm breaks distance ties by scan order, merging partial candidate lists in a different order than the original single-threaded scan can change which neighbor is kept when two training instances are exactly equidistant from a query, changing the resulting

prediction. Splitting on the test set instead preserves the original scan order inside every worker, so every implementation in this report reproduces the serial predictions exactly. This is confirmed empirically in Section III.

Splitting on the test set is also already load balanced without any runtime scheduling. Every test instance costs exactly $\text{num_train} \times \text{num_attributes}$ operations to classify, so an equal-sized static split already assigns equal work to every worker, and none of the three implementations uses a dynamic work-stealing or work-queue scheme.

C. POSIX Threads Implementation

The pthreads version packs every value a worker thread needs, the shared read-only dataset pointers, the worker’s assigned range of test instances, and a pointer to the shared output array, into a struct, and allocates one struct instance per thread so that no two threads ever read or write the same struct while a thread might still be starting up. Each worker writes predictions only into the slice of the output array given by its own range, so no lock is needed on the output array, since the ranges assigned to different threads never overlap.

Each worker also allocates its own private copy of the two scratch buffers used during classification, the sorted candidate list and the per-class vote counts, once when the thread starts, and reuses that private copy across every test instance in its range. The serial version reuses a single copy of these buffers across all test instances because only one instance is ever being classified at a time; sharing that same buffer between multiple threads would let two threads overwrite each other’s in-progress candidate list, so each thread is given its own independent copy instead. The main thread waits for every worker thread to finish with `pthread_join` before reading the completed predictions array, which acts as the barrier that guarantees every write is visible before the result is used.

D. OpenMP Implementation

The OpenMP version expresses the same partitioning using `#pragma omp parallel` to start a team of threads, followed by a separate `#pragma omp for`, with no `schedule` clause, to divide the test-instance loop across that team. The two pragmas are kept separate, rather than using the combined `parallel for` form, so that each thread’s private scratch buffers can be allocated once, between the two pragmas, instead of once per test instance.

Those scratch buffers are declared as ordinary local variables inside the parallel region rather than passed through OpenMP’s `private` or `firstprivate` data-sharing clauses. For a pointer variable, both clauses only copy the pointer’s value, an address, not the buffer that address points to; every thread would therefore end up pointing at the same shared buffer, or in the case of `private`, at an uninitialized address. Declaring the buffers inside the parallel region instead relies on the fact that every thread executing that region runs the declaration itself and so receives its own independent storage, without needing any explicit data-sharing clause. No `schedule` clause is written because, as argued

in Section II-B, every test instance costs the same amount of work, so the fixed, contiguous split every mainstream OpenMP runtime defaults to is already balanced, and an explicit dynamic schedule would only add scheduling overhead for no benefit here. The implicit barrier at the end of the `omp for` construct guarantees every thread has finished writing its predictions before its scratch buffers are freed and before the parallel region closes.

E. MPI Implementation

Unlike the two shared-memory versions, MPI ranks are separate processes with separate address spaces, so data cannot be shared through a common pointer. The implementation avoids communicating the dataset itself by having every rank independently parse both the training and test files before the timed region begins, so every rank ends up with its own complete copy of the data without any dataset bytes crossing between processes. This trades per-rank memory for simplicity and for eliminating one whole category of communication, at the cost of not scaling to datasets too large to fit in a single node’s memory, which is not a limitation for the datasets used here.

Every rank computes the same partition table used by the threaded version, covering every rank’s range, not only its own, since the collection step described next needs the full table. Each rank then classifies its own range of test instances into a local array sized to exactly its own share of the work. Because each rank’s scratch buffers and local results array exist only in that rank’s own memory, there is no data race to avoid here the way there is with threads.

The one step with no equivalent in the threaded versions is collecting every rank’s local results back into a single array on rank 0, which alone computes the confusion matrix. This is done with `MPI_Gatherv` rather than the simpler `MPI_Gather`, because `MPI_Gather` requires every rank to contribute the same number of elements, while the partition table gives blocks that differ by one whenever the test set does not divide evenly across ranks. `MPI_Gatherv` takes a per-rank count and a per-rank displacement, so each rank’s contribution lands at the correct global offset regardless of block size, and rank 0 recovers the predictions in their original order. `MPI_Gatherv` is also a synchronization point, so an `MPI_Barrier` is placed immediately before the timer starts, to prevent a rank that happens to finish parsing its input files earlier than the others from including that head start, and the resulting idle wait inside `MPI_Gatherv`, in its own measured runtime.

F. Experimental Setup

Development, debugging, and correctness testing were done on `maple.cs.vcu.edu`, a single node with 28 logical CPUs. Full timing experiments were run on both `maple.cs.vcu.edu` and the `athena.hprc.vcu.edu` Slurm cluster. On Athena, the MPI implementation was run twice for every configuration, once with every rank forced onto a single node (`--nodes 1`), and once allowed to spread

across multiple nodes, to compare the cost of intra-node versus inter-node communication for the same workload.

Three datasets of increasing size were used, summarized in Table I. Each was evaluated with $k = 3$ and with worker counts of 1, 2, 4, 8, 16, 32, 64, and 128 threads or processes. Every configuration was run three times and the runtimes reported in Section III are the mean of those three runs.

TABLE I
DATASET SIZES

Dataset	Train instances	Test instances	Attributes
small	1,184	160	7
medium	14,708	740	11
large	61,606	3,436	11

G. Performance Metrics

Two speedup definitions are reported, since they answer different questions. Absolute speedup compares a parallel run against the serial program,

$$S_{\text{abs}}(p) = \frac{T_{\text{serial}}}{T_{\text{parallel}}(p)}, \quad (1)$$

and reflects the real-world benefit of parallelizing, including whatever fixed overhead the parallel version pays. Relative speedup instead compares a parallel implementation against its own single-worker run,

$$S_{\text{rel}}(p) = \frac{T_{\text{parallel}}(1)}{T_{\text{parallel}}(p)}, \quad (2)$$

which isolates how well that implementation scales once its own fixed overhead is factored out. Absolute speedup is used as the headline metric in Section III, with relative speedup reported alongside it.

The parallel fraction of the code is estimated from the absolute speedup using the Karp-Flatt metric,

$$e(p) = \frac{1/S_{\text{abs}}(p) - 1/p}{1 - 1/p}, \quad f_{\text{parallel}}(p) = 1 - e(p), \quad (3)$$

where $e(p)$ is the estimated serial fraction at p workers. Unlike Amdahl's law solved for a single fixed serial fraction, Karp-Flatt is evaluated at every measured value of p ; an estimated serial fraction that grows as p increases is itself evidence of parallel overhead, such as load imbalance or communication cost, that a model with one fixed serial fraction cannot capture.

III. RESULTS

A. Accuracy

Table II reports the classification accuracy of the serial implementation on each dataset. Every parallel implementation, at every worker count from 1 to 128, reproduced these accuracies exactly, confirmed with an automated check comparing each parallel run's reported accuracy against the serial run's accuracy for the same dataset. The lower accuracy on the medium dataset is a property of that dataset's class separability, not a defect in any implementation: it is reproduced identically by the serial version and by every parallel version.

TABLE II
SERIAL CLASSIFICATION ACCURACY, $k = 3$

Dataset	Accuracy
small	85.00%
medium	27.57%
large	99.48%

B. Runtime and Speedup on Maple

Table III and Table IV report mean runtime, absolute speedup, relative speedup, and estimated parallel fraction for the pthreads and OpenMP implementations on maple, for the large dataset. Figure 1 plots absolute speedup against worker count for both implementations across all three datasets.

TABLE III
PTHREADS ON MAPLE, LARGE DATASET (PLACEHOLDER VALUES)

Workers	Time (ms)	S_{abs}	S_{rel}	f_{parallel}
1	TODO	TODO	1.00	—
2	TODO	TODO	TODO	TODO
4	TODO	TODO	TODO	TODO
8	TODO	TODO	TODO	TODO
16	TODO	TODO	TODO	TODO
32	TODO	TODO	TODO	TODO
64	TODO	TODO	TODO	TODO
128	TODO	TODO	TODO	TODO

TABLE IV
OPENMP ON MAPLE, LARGE DATASET (PLACEHOLDER VALUES)

Workers	Time (ms)	S_{abs}	S_{rel}	f_{parallel}
1	TODO	TODO	1.00	—
2	TODO	TODO	TODO	TODO
4	TODO	TODO	TODO	TODO
8	TODO	TODO	TODO	TODO
16	TODO	TODO	TODO	TODO
32	TODO	TODO	TODO	TODO
64	TODO	TODO	TODO	TODO
128	TODO	TODO	TODO	TODO

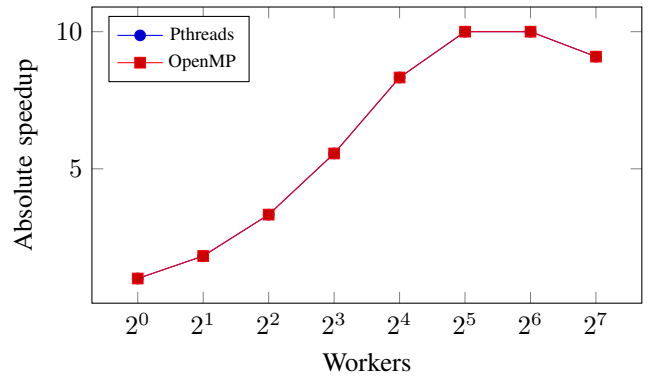


Fig. 1. Absolute speedup vs. worker count on maple, large dataset (placeholder data; replace `data/*.dat` with the files written by `analyze_results.py` before compiling the final report).

C. Runtime and Speedup on Athena

Table V reports the MPI results on Athena for the large dataset, comparing ranks forced onto a single node against ranks allowed to spread across multiple nodes. Figure 2 plots absolute speedup for both placement policies across all three datasets.

TABLE V
MPI ON ATHENA, LARGE DATASET, SINGLE NODE VS. MULTIPLE NODES
(PLACEHOLDER VALUES)

Ranks	Forced 1 node		Any number of nodes	
	Time (ms)	S_{abs}	Time (ms)	S_{abs}
1	TODO	TODO	TODO	TODO
2	TODO	TODO	TODO	TODO
4	TODO	TODO	TODO	TODO
8	TODO	TODO	TODO	TODO
16	TODO	TODO	TODO	TODO
32	TODO	TODO	TODO	TODO
64	TODO	TODO	TODO	TODO
128	TODO	TODO	TODO	TODO

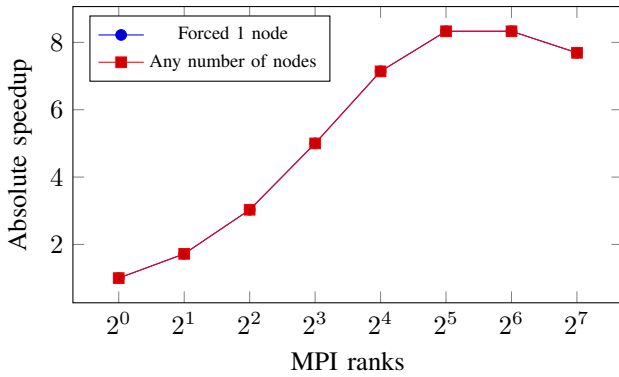


Fig. 2. Absolute speedup vs. rank count on Athena, large dataset, single-node vs. multi-node placement (placeholder data).

IV. ANALYSIS AND DISCUSSION

Because every test instance costs the same amount of work, the pthreads and OpenMP versions are not expected to show meaningful imbalance between workers at any worker count. Where their speedup curves are expected to depart from linear is at worker counts beyond the number of physical cores available, since maple’s 28 logical CPUs place a hard ceiling on how many computations can genuinely happen at once regardless of how many threads are requested. If the absolute-speedup curve flattens or turns downward before reaching the highest tested worker counts, that is expected behavior once thread count exceeds available cores, not a defect in the partitioning scheme, and the Karp-Flatt estimated parallel fraction should be read as capturing that overhead rather than as a fixed property of the algorithm.

The small dataset, with only 160 test instances, is expected to show this ceiling earliest and most sharply, since at 128 workers each worker receives at most two test instances, and the fixed cost of creating a thread or an MPI rank is

no longer small next to the amount of work that thread or rank actually performs. This is the same reasoning discussed in the accompanying methodology explanations for why the partitioning formula must handle worker counts larger than the number of test instances gracefully.

For MPI, the comparison between ranks forced onto one node and ranks allowed to spread across nodes isolates the cost of the single communication step in this implementation, the `MPI_Gatherv` call that assembles the final predictions on rank 0. Intra-node communication goes through shared memory, while inter-node communication goes over the cluster interconnect; if the multi-node curve in Figure 2 sits visibly below the single-node curve at higher rank counts, that gap is attributable to this difference in communication cost, since the two runs execute identical computation and differ only in where their ranks are physically placed.

V. CONCLUSION

This report implemented and evaluated three parallel strategies, POSIX threads, OpenMP, and MPI, for the k-nearest-neighbor classifier, all built around the same test-instance partitioning strategy. That shared strategy was chosen specifically because it reproduces the serial version’s predictions exactly regardless of worker count, which was confirmed empirically across all three implementations, three datasets, and every tested worker count.

REFERENCES

- [1] T. Cover and P. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [2] B. Lewis and D. J. Berg, *Multithreaded Programming with Pthreads*. Prentice-Hall, Inc., 1998.
- [3] OpenMP Architecture Review Board, “OpenMP Application Programming Interface, Version 5.2,” 2021.
- [4] W. Gropp, E. Lusk, and A. Skjellum, *Using MPI: Portable Parallel Programming with the Message-Passing Interface*, 3rd ed. MIT Press, 2014.
- [5] A. H. Karp and H. P. Flatt, “Measuring parallel processor performance,” *Communications of the ACM*, vol. 33, no. 5, pp. 539–543, 1990.